

Universidad ORT Uruguay
Facultad de Ingeniería

Diseño de Aplicaciones II
Obligatorio 2

Documentación

Santiago Duró - 256325 - M6C

Lucas Facello - 298679 - M6C

Federico Katz - 257073 - M6B

Docentes M6C: Nicolás Fierro, Alexander Wieler
Docentes M6B: Daniel Acevedo, Federico González

Repositorio:

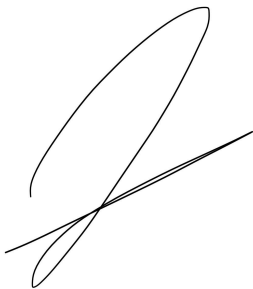
https://github.com/IngSoft-DA2/257073_256325_298679

21 de Noviembre, 2024

DECLARACIÓN DE AUTORÍA

Nosotros, Santiago, Federico, y Lucas, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el curso de Diseño de Aplicaciones II;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- En la obra, hemos acusado recibo de las ayudas recibidas;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Santiago Duró

21 de noviembre 2024



Lucas Facello

21 de noviembre 2024



Federico Katz

21 de noviembre 2024

Índice

Descripción del trabajo.....	4
Descripción general.....	4
Descripción del sistema.....	4
Errores Conocidos / Dudas Técnicas.....	5
Diagramas.....	6
Vista de Desarrollo.....	6
Diagrama de Paquetes.....	6
Diagrama de componentes.....	7
Vista Lógica.....	8
Diagrama de CargarImplementacion.....	10
Diagrama de Persistencia.....	10
Diagrama de WebApi.....	11
Vista de Procesos.....	12
Diagramas de Secuencia.....	12
Modelo de Tablas.....	15
Justificación y Explicación de Diseño.....	16
Principios de Diseño.....	17
Patrones de Diseño.....	19
→ Adapter:.....	19
→ Strategy con Fábrica:.....	20
Mecanismos utilizados para la extensibilidad solicitada.....	22
Métricas.....	24
Mejoras a destacar con respecto a la primer entrega.....	26
Anexo.....	28
Descripción de los resources de la API.....	28
Administradores.....	28
Cámaras.....	28
Cuartos.....	29
Dispositivos.....	29
Dispositivos Hogar.....	30
Dueños Empresa.....	31
Dueños Hogar.....	31
Empresas.....	32
Hogares.....	33
Importadores.....	36
Lámparas.....	37
Notificaciones.....	37
Sensores.....	38
Sesiones.....	39
Usuarios.....	39
Cobertura de Test Unitarios.....	41
Instalación.....	43

Descripción del trabajo

Descripción general

Este proyecto fue realizado utilizando los entornos de desarrollo Rider y Visual Studio, como también GitHub para el versionado del mismo. El lenguaje utilizado fue .NetCore versión 8.0. Para la gestión de la base de datos, se empleó Microsoft SQL Server Management Studio 2022.

Para esta entrega, también se incorporó el uso de Angular para el desarrollo del frontEnd, desarrollada en el entorno Visual Studio Code. Además, se utilizó Bootstrap como framework de diseño para la interfaz de usuario. Además, el proyecto incorpora el uso de Reflection como parte de su implementación.

Descripción del sistema

En esta segunda entrega, el proyecto continúa enfocándose en la centralización y gestión de dispositivos inteligentes para hogares, ampliando sus funcionalidades. El sistema ahora integra una interfaz de usuario (UI) a través de una aplicación Angular (SPA) que permite implementar todas las características descritas en la entrega pasada y las nuevas requeridas.

La aplicación sigue contando con tres roles principales de usuario: administrador, dueño de empresa y dueño de hogar, cada uno con una lista específica de permisos. Los administradores y dueños de empresa, además, pueden adoptar el rol de dueño de hogar sin perder sus permisos originales, permitiéndoles operar como cualquier otro miembro del hogar.

El sistema se ha enriquecido con múltiples funcionalidades nuevas que mejoran tanto la usabilidad como la personalización. Ahora, los usuarios pueden asignar nombres personalizados a sus hogares y dispositivos, lo que facilita su identificación y organización sin interferir en otros contextos. Además, se han incorporado nuevos tipos de dispositivos, como sensores de movimiento y lámparas inteligentes, ampliando las capacidades de automatización y control dentro del hogar.

Se ha introducido la posibilidad de agrupar dispositivos por habitaciones, permitiendo gestionar y visualizar dispositivos en función de su ubicación. Asimismo, el listado de dispositivos ha sido mejorado para incluir información detallada sobre su estado y ofrecer filtros más específicos, como por habitación. Estas nuevas características hacen que el sistema sea más intuitivo y adaptable para los usuarios.

Además, se han implementado validaciones personalizables para los modelos de dispositivos en el registro de empresas. Cada empresa puede definir qué lógica de validación aplicar, utilizando una interfaz estándar que asegura la extensibilidad del sistema.

Un aspecto innovador es la incorporación de importadores dinámicos para dispositivos inteligentes. Esto permite a terceros integrar nuevos formatos de importación (como XML, JSON, bases de datos, o servicios HTTP) en tiempo de ejecución, sin necesidad de recompilar el sistema. Esta flexibilidad responde a la necesidad de internacionalizar y ampliar el ecosistema del sistema de manera escalable.

Como en la primera entrega, el diseño del proyecto sigue una estructura vertical, abordando cada recurso o funcionalidad de manera incremental. Este enfoque permite garantizar la estabilidad e integración de cada nueva característica antes de avanzar. La metodología TDD se mantiene como base, asegurando la calidad del código y la cobertura de tests en todas las áreas del sistema.

Finalmente, el backend de la solución ahora se conecta de manera fluida con el frontend desarrollado en Angular, permitiendo probar y utilizar todas las funcionalidades en un entorno visual amigable para el usuario. El enfoque en clean code y los principios SOLID siguen guiando las decisiones de diseño y desarrollo del equipo, lo que asegura un código limpio, modular y extensible.

Errores Conocidos / Dudas Técnicas

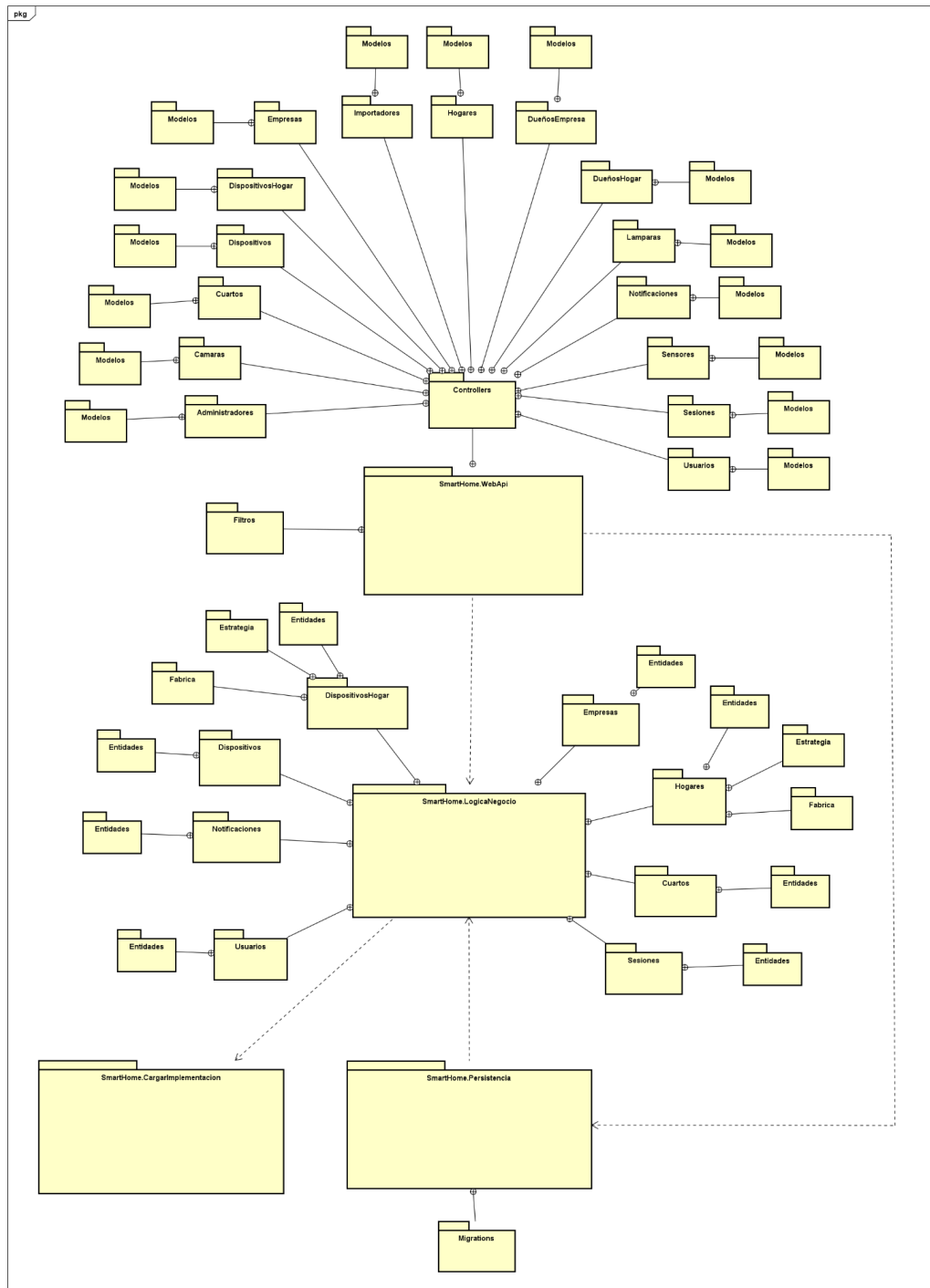
- Uno de los problemas identificados en esta entrega es la falta de consistencia en los idiomas utilizados entre el backend y el frontend. Mientras que el backend está desarrollado en español, el frontend utiliza inglés, lo que puede generar confusión en la nomenclatura y complicar la comunicación entre ambas capas del sistema.
- Otro aspecto a destacar es la ausencia de un paquete de abstracciones independiente. Aunque esto se había recomendado como una buena práctica para mejorar la modularidad y la extensibilidad del sistema, la estructura vertical adoptada para el desarrollo hizo que todo el dominio y la lógica de negocio quedarán consolidados en un único paquete denominado LogicaNegocio. Esta decisión, si bien favoreció el desarrollo incremental, dificulta la separación de responsabilidades y podría limitar futuras expansiones del sistema.
- Si bien se realizaron esfuerzos para delegar validaciones y otras responsabilidades que antes se realizaban en los controladores a la lógica de negocio, algunos controladores pueden contener algo de lógica que no corresponde a su propósito, aunque en menor medida que en la entrega anterior.
- La falta de excepciones personalizadas también es un punto que podríamos haber mejorado. Actualmente usamos 4 tipos diferentes de excepciones, pero no creamos ninguna puntual.

Diagramas

Uso del modelo 4 + 1 para documentar la arquitectura de nuestro sistema.

Vista de Desarrollo

Diagrama de Paquetes



Paquete WebApi: Actúa como la capa de presentación, recibiendo solicitudes HTTP, procesándolas y devolviendo respuestas a los clientes. Los Controllers gestionan las solicitudes del cliente y se comunican con la lógica de negocio. Los Servicios se utilizan para implementar lógica común que pueda ser compartida entre varios controladores. Los Filtros gestionan la interceptación de solicitudes o respuestas para aplicar validaciones o autenticación.

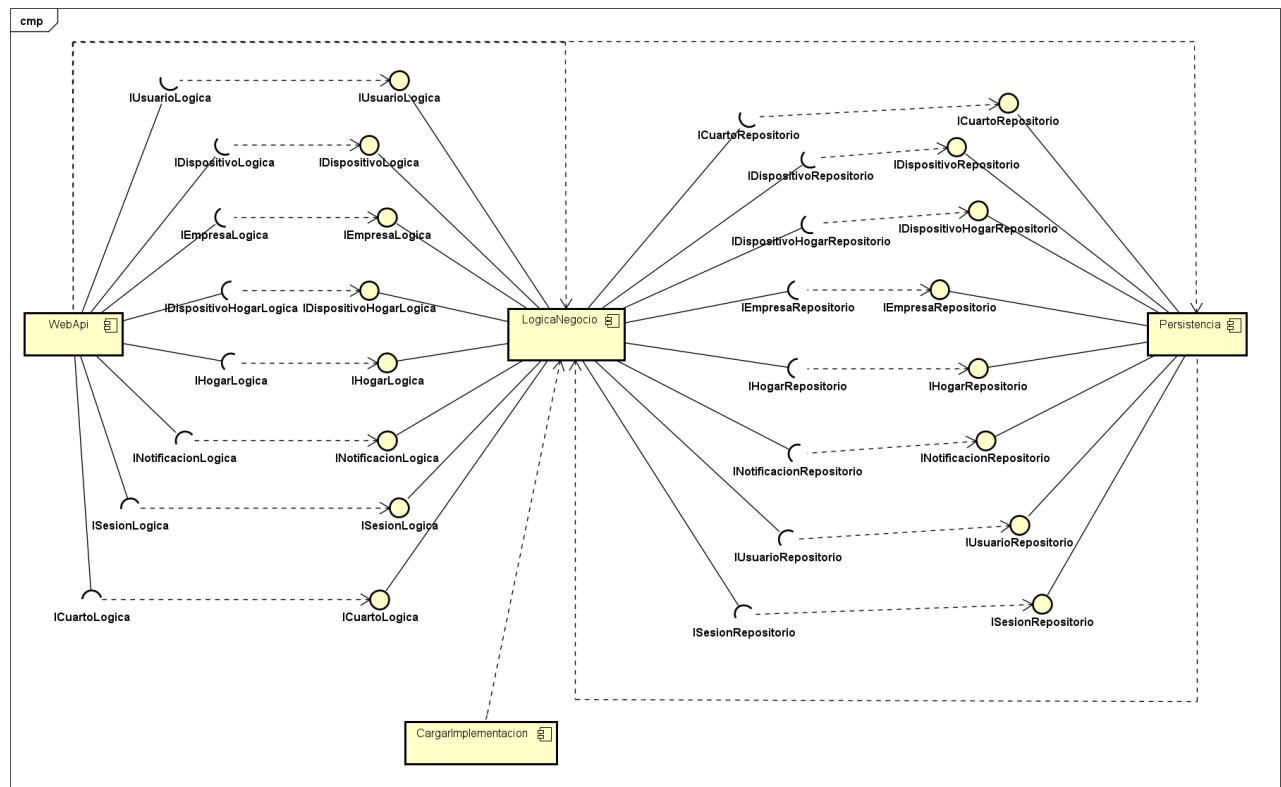
Paquete LogicaNegocio: Implementa toda la lógica y reglas de negocio del sistema, aplicando las reglas específicas para cada funcionalidad. Las clases dentro de este paquete se encargan de gestionar dispositivos, hogares, usuarios, notificaciones y empresas.

Dentro de este paquete optamos por incluir tanto los servicios como el dominio, aumentando la cohesión y disminuyendo el acoplamiento. Utilizamos una arquitectura vertical para fomentar el reuso de las features.

Paquete Persistencia: Gestiona el acceso a la base de datos y la persistencia de los objetos. Utiliza repositorios para abstraer el acceso a la base de datos, garantizando que la lógica de negocio no dependa de implementaciones específicas de almacenamiento.

Paquete CargarImplementacion: El objetivo del paquete es proporcionar un mecanismo dinámico para la carga e instanciación de implementaciones concretas de interfaces a partir de ensamblados externos (DLLs). Este paquete está diseñado para permitir una arquitectura extensible, fomentando la modularidad y la flexibilidad del sistema, haciendo posible adaptarse a nuevas necesidades o integraciones específicas de terceros.

Diagrama de componentes

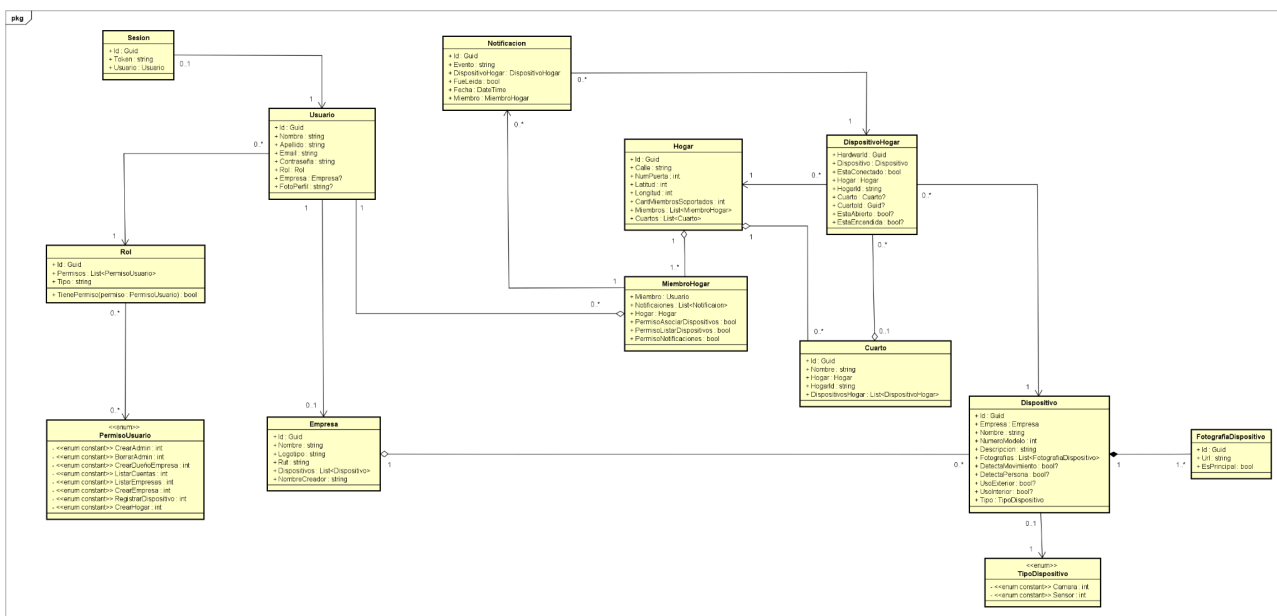


Decidimos implementar estos componentes porque el sistema fue diseñado para operar a través de capas interconectadas, alineadas con el principio de separación de responsabilidades. Este enfoque, combinado con el uso del patrón de inyección de dependencias, nos ofrece múltiples ventajas:

- Independencia entre capas: Cada capa, desde la Web API, pasando por la Lógica de Negocio, hasta la Persistencia, funciona de manera autónoma y está claramente definida, lo que reduce el acoplamiento y mejora la modularidad.
- Mantenibilidad y comprensión: Al cumplir con el principio de responsabilidad única, donde cada componente se centra en una función específica, el sistema es más fácil de entender. Por ejemplo, las reglas de negocio están aisladas en la capa de Lógica de Negocio, mientras que la gestión de datos se maneja en la capa de Persistencia. Esto simplifica la detección y resolución de errores.
- Facilidad para gestionar cambios: Aunque algunos cambios pueden requerir modificaciones en más de una capa, como ajustar la lógica de negocio y su correspondiente repositorio en la persistencia, esta arquitectura localiza los cambios, minimizando el impacto en el resto del sistema. Por ejemplo, si necesitamos agregar una nueva validación para dispositivos, esta puede implementarse únicamente en la capa de Lógica de Negocio, sin afectar a la Persistencia ni a la Web API.

Este diseño permite trabajar de manera eficiente, ya que cualquier modificación está contenida en las capas específicas relacionadas con la funcionalidad. Así, se garantiza un mantenimiento más ágil y menos propenso a errores.

Vista Lógica



Dentro del dominio de nuestro sistema podemos encontrar todas las clases utilizadas. En este punto nos parece relevante destacar ciertos aspectos que tuvimos en cuenta a la hora de diseñar nuestro dominio.

Luego de discutir, decidimos hacer una clase Usuario, y que uno de los atributos de la misma sea otra clase Rol. En esta clase tenemos un Id hardcodeado por defecto para cada uno de los roles pedidos para esta entrega, ya que si dejáramos que el identificador sea autogenerado, lo que pasaría es que cada vez que se genere una nueva migración va a querer actualizar la seed data que está trackeando, lo que imposibilita que los roles se relacionen con otras tuplas generadas en tiempo de ejecución. Por último, decidimos implementar una Lista de Permisos, donde se especifica cuales son las acciones que puede realizar cada uno de los roles del sistema. Esta última clase la definimos como un enum, donde tenemos todas las funcionalidades que pueden llevar a cabo los usuarios.

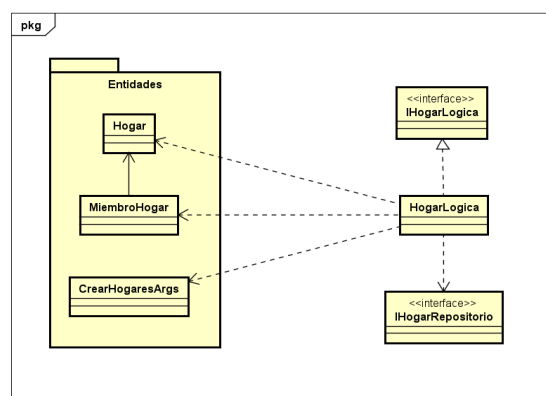
RolesPredefinidos	
+ ID_DUEÑO_HOGAR :	Guid
+ ID_ADMIN :	Guid
+ ID_DUEÑO_EMPRESA :	Guid
+ ID_ADMIN_DUEÑO_HOGAR :	Guid
+ ID_DUEÑO_EMPRESA Y HOGAR :	Guid
+ DueñoHogar :	Rol
+ Admin :	Rol
+ DueñoEmpresa :	Rol
+ AdminDueñoHogar :	Rol
+ DueñoEmpresaYHogar :	Rol

La forma en la que manejamos los dispositivos asociados a un hogar. Decidimos crear una clase intermediaria entre Dispositivo y Hogar, donde guardamos el dispositivo que se asocia y el hogar al que está asociado.

También utilizamos el mismo razonamiento para manejar los miembros asociados al hogar, con una tabla intermedia entre Usuario y Hogar.

Para cada feature dentro de la lógica del negocio, utilizamos una estructura similar: una carpeta con las clases del dominio que serán implementadas por dicha feature, y los archivos con la lógica, la interfaz de la lógica y la interfaz del repositorio.

Si lo llevamos al caso concreto de la feature Hogares (para el resto de features, se utiliza el mismo razonamiento), nos encontraríamos con el siguiente diagrama:



En cuanto a los métodos que debe implementar cada lógica concreta, estos son definidos por sus respectivas interfaces:

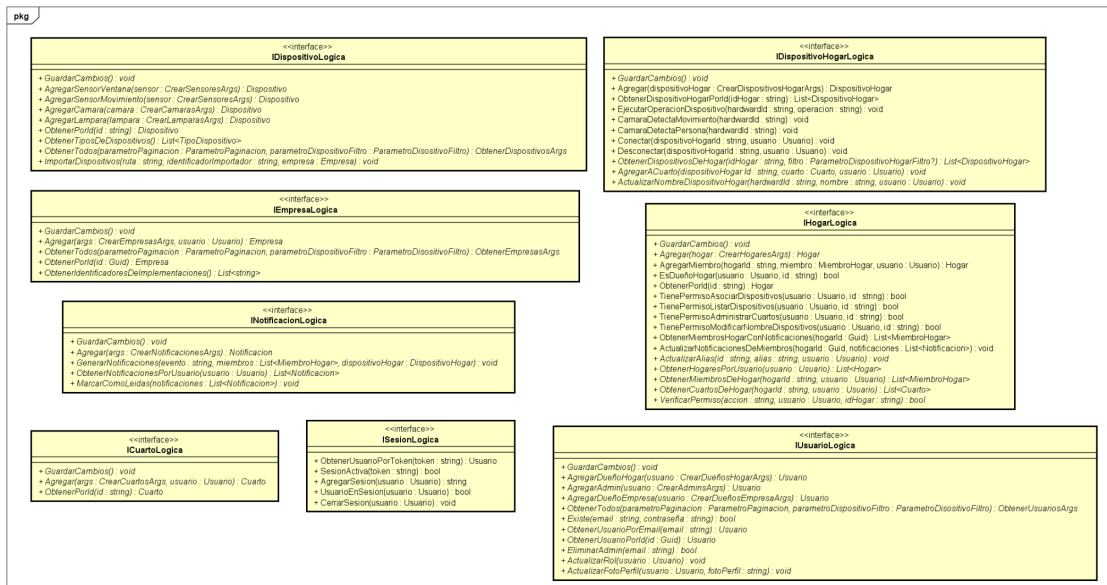
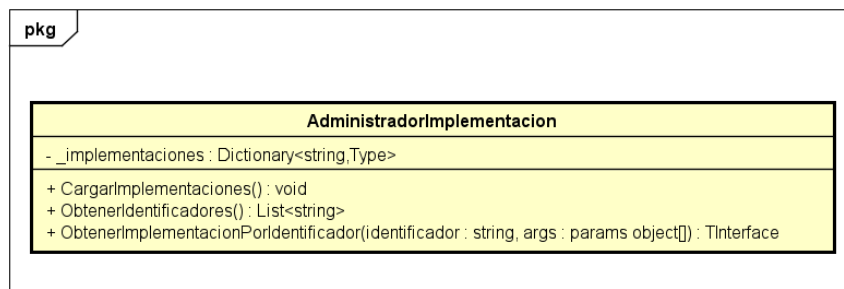


Diagrama de CargarImplementacion



Contiene una única clase, justificada por su responsabilidad aislada y poco cohesiva con otros paquetes. Al mantener esta implementación aislada, el paquete comunica de manera explícita que su objetivo es proporcionar una funcionalidad específica, sin necesidad de interactuar con el resto del sistema de manera directa.

Diagrama de Persistencia

Manejamos una clase ContextoSql que contiene todos los sets con nuestras clases para representar las tablas correspondientes dentro de la base de datos.

Los repositorios contienen todos aquellos métodos necesarios para poder interactuar con la base de datos de manera efectiva.

Utilizamos una relación de composición, ya que si el ContextoSql se destruye o deja de existir, el repositorio no puede seguir funcionando porque depende totalmente de esa

instancia para realizar sus operaciones. La composición refleja una dependencia mucho más fuerte que una asociación. En este caso, el repositorio no puede funcionar sin el contexto, lo cual es una característica clave de la composición.

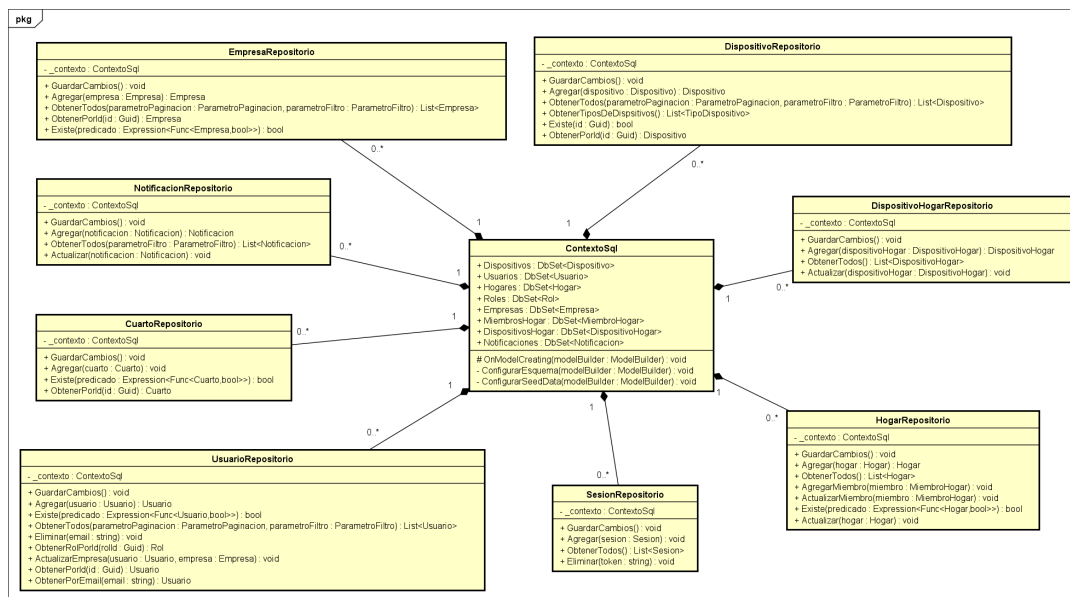
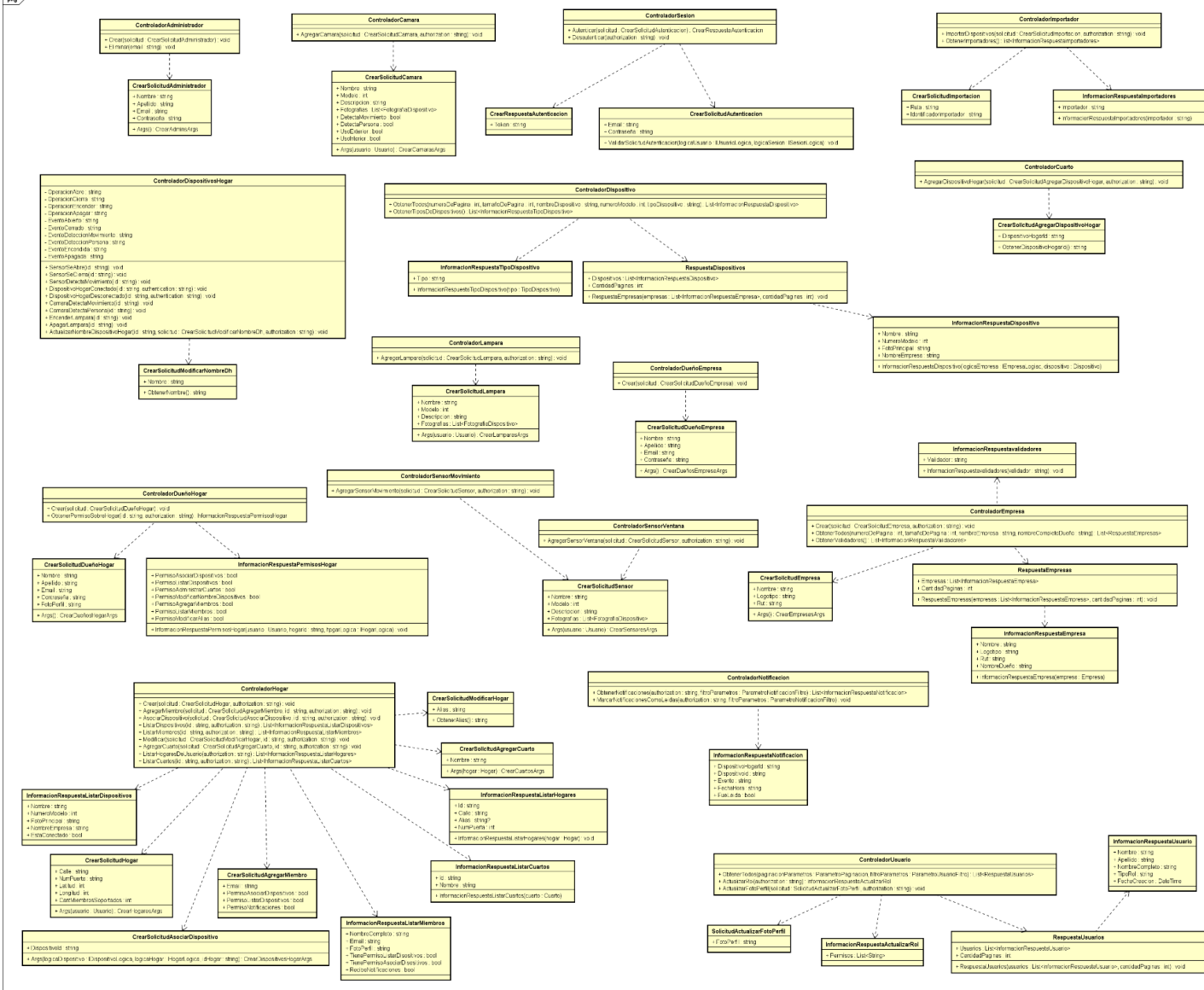


Diagrama de WebApi

En el paquete están definidos todos los controladores y los modelos de entrada y salida. Los modelos nos sirven para no exponer nuestras clases de la lógica mejorando la integridad, y realizan validaciones para evitar el ingreso de datos inválidos a nuestra lógica.. Además, al crear una nueva entidad solo debemos especificar los campos requeridos por el modelo, por lo que si cambia el dominio, el mismo no afectará a las aplicaciones que consuman nuestra API.

Dentro de WebApi también podemos encontrar los filtros de autenticación (comprueba que el usuario esté logueado), autorización (comprueba que el usuario tenga el permiso necesario para realizar una operación) y excepciones (mediante un diccionario que asigna comportamientos a excepciones específicas).

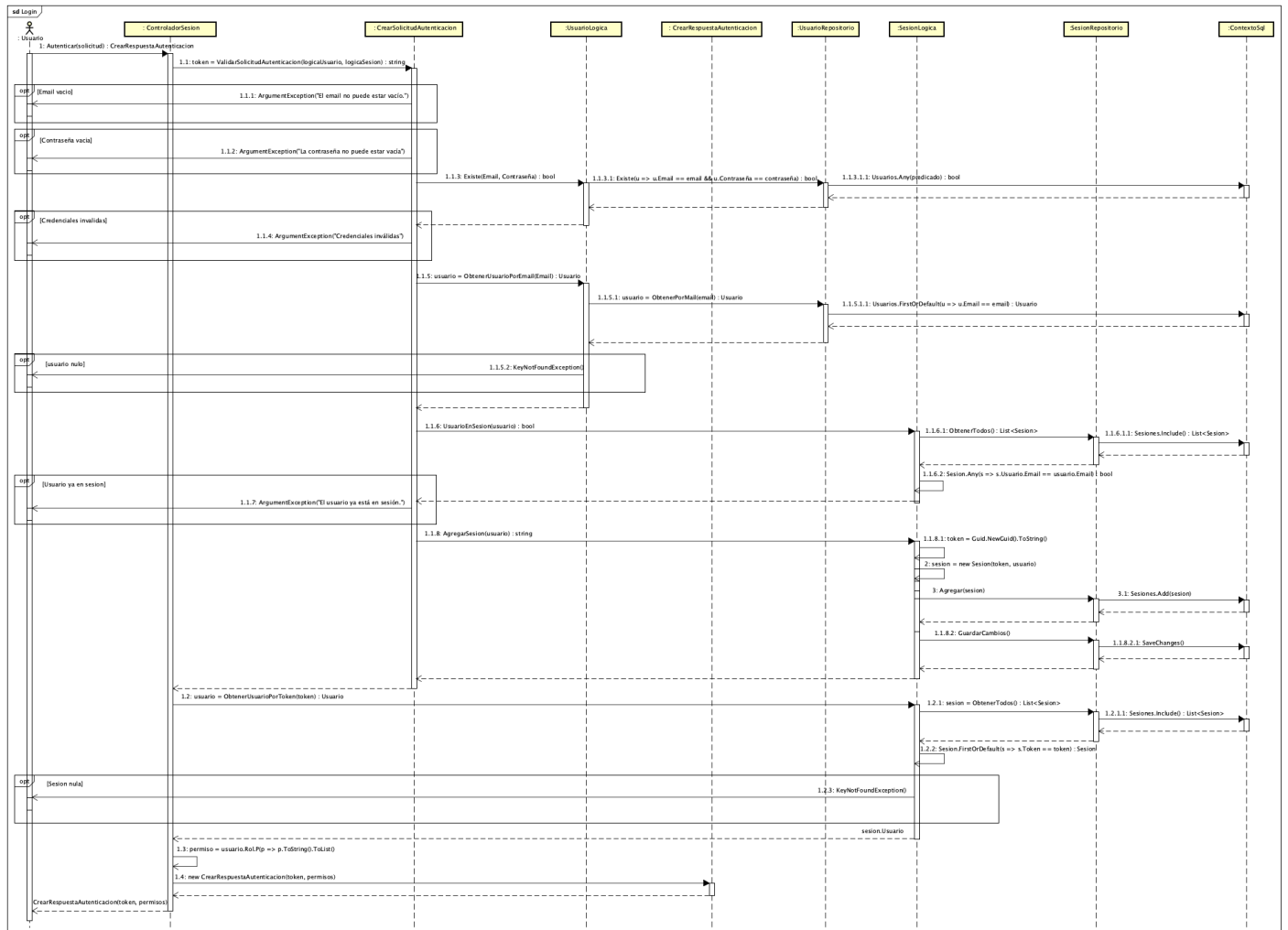


Vista de Procesos

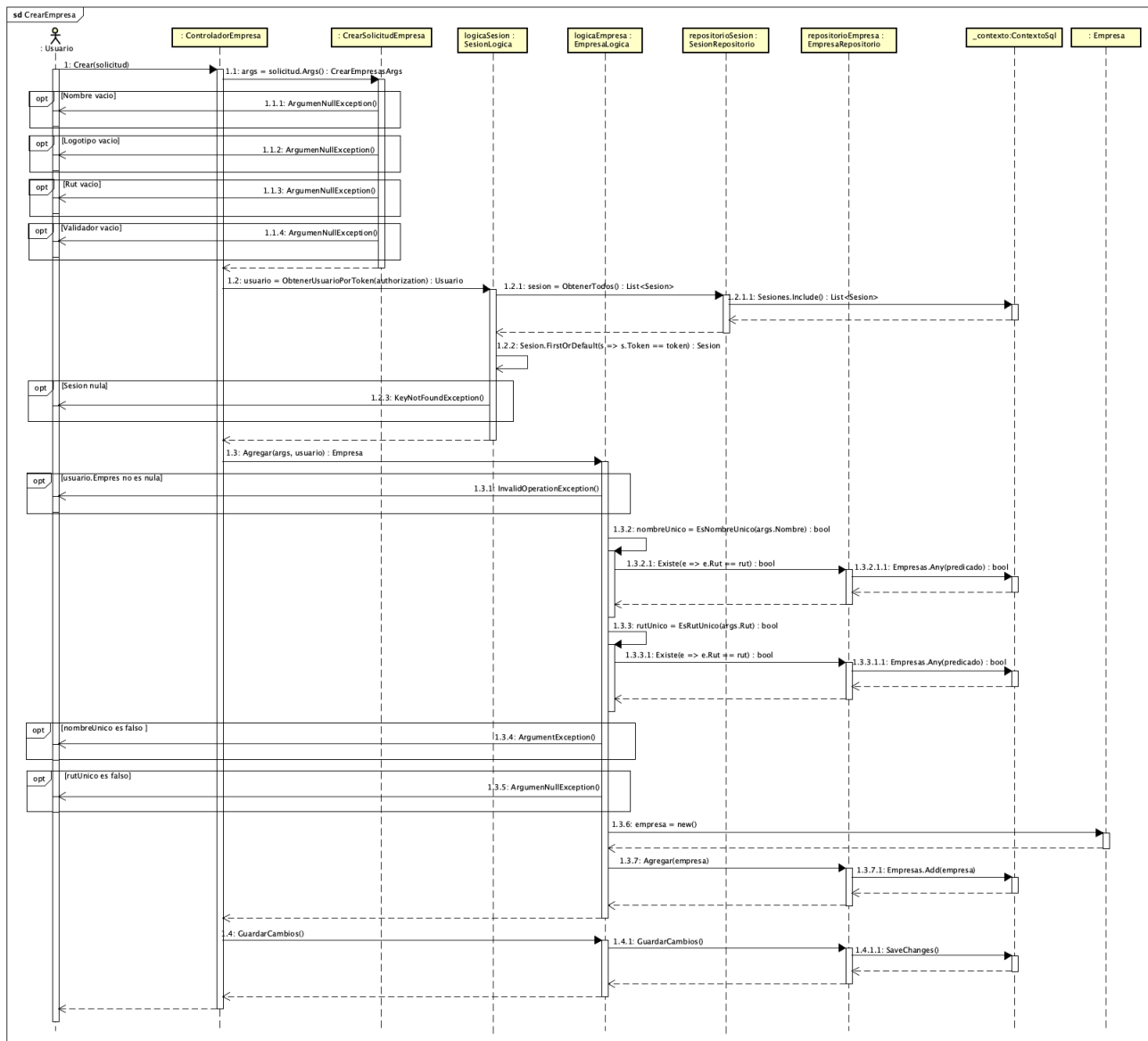
Diagramas de Secuencia

En esta sección, veremos los diagramas de secuencia de algunas funcionalidades de nuestra aplicación, donde se puede ver cómo los distintos componentes del sistema funcionan en conjunto para llevar a cabo las principales funcionalidades del mismo. Nos centramos en seleccionar algunas de las funcionalidades que consideramos de crítica importancia para nuestro sistema, como lo pueden ser el login exitoso de un usuario, la creación de una empresa y añadir un miembro a un hogar.

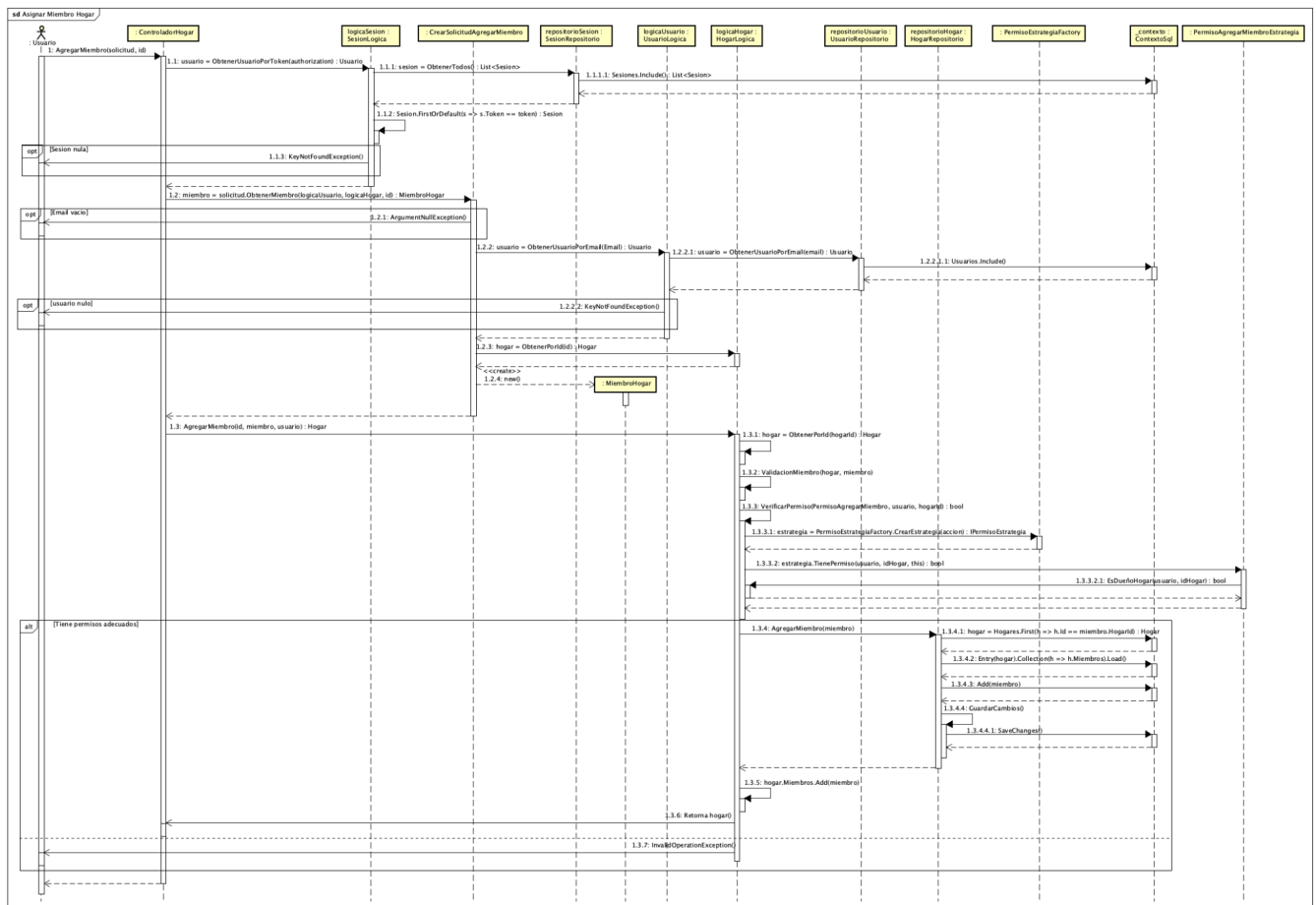
Login exitoso: Muestra los pasos necesarios para autenticar a un usuario en el sistema. Con respecto a la primera entrega, hay ciertos cambios en el funcionamiento del logIn a la aplicación. En comparación con el diagrama anterior, notamos que ahora las Sesiones son persistentes, por eso se agrandó considerablemente el diagrama. También se agregaron todas las validaciones llevadas a cabo a lo largo de esta funcionalidad. Estos arreglos se ven reflejados en el diagrama a continuación.



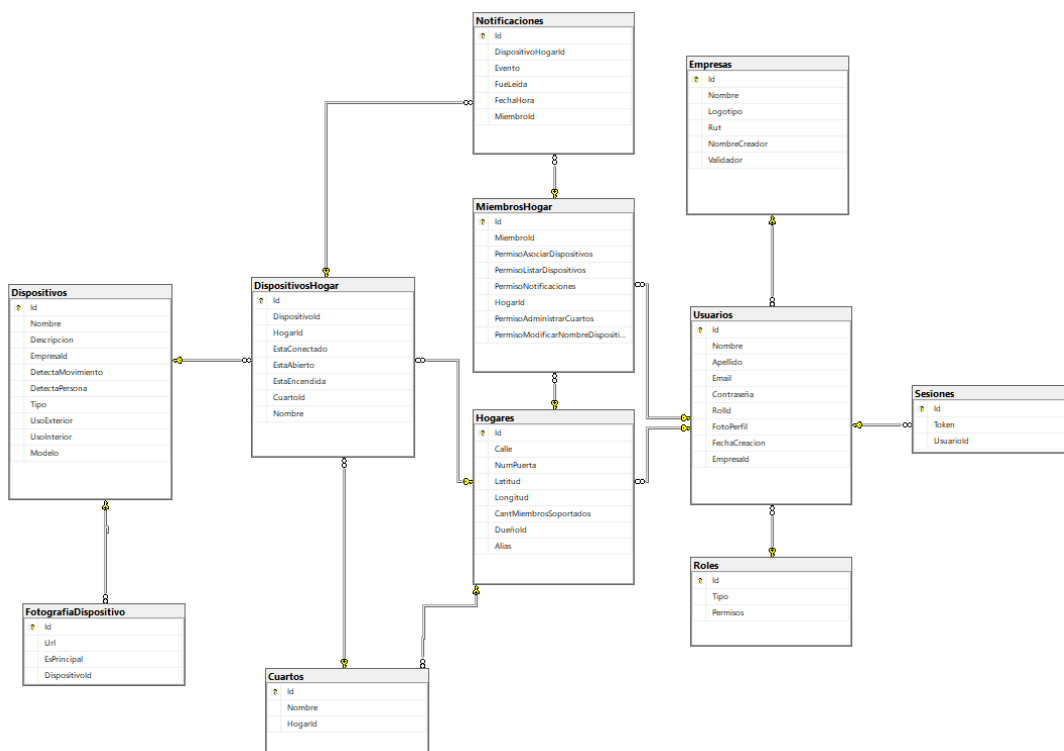
Creación de empresa exitosa: Detalla el proceso de creación de una nueva empresa, validando que no exista ninguna en el sistema con el mismo nombre o rut. Este diagrama también fue actualizado con respecto a la primera entrega. Se agregaron varios opt donde se lanzan excepciones, como también cambios dentro de la lógica.



Añadir miembro a hogar exitoso: Representa el flujo necesario para añadir un nuevo miembro a un hogar existente. Para esta entrega, decidimos modificar este diagrama también. Hubo modificaciones en el controlador de Hogar y en la lógica de Hogares. Sacamos ciertas validaciones que se llevaban a cabo dentro del paquete WebApi para evitar romper el SRP. Aclaramos que en el diagrama, no representamos los métodos ObtenerPorId (dentro de la lógica), ya que se repite varias veces y no aporta información relevante al flujo que se busca destacar.



Modelo de Tablas



Este diagrama muestra el esquema actual de la base de datos generado por Entity Framework utilizando la metodología Code First. Las relaciones entre las tablas han sido correctamente definidas y reflejan la estructura lógica de nuestro sistema. Luego de realizar las migraciones correspondientes, esta es la versión actual del esquema.

Justificación y Explicación de Diseño

En el diseño de nuestro programa optamos por una estructura vertical, dividiendo los proyectos en LogicaNegocio, Persistencia y WebApi. Esta decisión se basó en principios de modularidad, separación de responsabilidades y escalabilidad. En esta arquitectura, LogicaNegocio centraliza las reglas de negocio, la lógica específica de cada entidad y todas las validaciones necesarias, asegurando la integridad de las operaciones. Persistencia se encarga exclusivamente del acceso a la base de datos, desacoplando los detalles de almacenamiento de la lógica. Por último, WebApi actúa como intermediario entre los clientes y las capas internas, limitándose a manejar las solicitudes y delegar el procesamiento a LogicaNegocio. Esta organización facilita el mantenimiento, promueve el desarrollo incremental y permite una navegación clara del código al agrupar los elementos relacionados en un mismo contexto.

En esta entrega, incorporamos un nuevo paquete llamado CargarImplementaciones, cuyo propósito principal es ofrecer un mecanismo dinámico para localizar e instanciar implementaciones concretas de interfaces desde ensamblados externos (DLLs). Este diseño tiene como objetivo facilitar una arquitectura más flexible y modular, permitiendo que el sistema sea fácilmente ampliable y adaptable. Gracias a este enfoque, es posible integrar nuevas funcionalidades o trabajar con implementaciones específicas de terceros sin necesidad de realizar modificaciones significativas en el código base, lo que fomenta la escalabilidad y la versatilidad del sistema.

Las ventajas que encontramos a la hora de utilizar esta estructura fueron las siguientes:

- *Escalabilidad*: La estructura modular permite que el sistema crezca sin comprometer el diseño actual. Por ejemplo, se pueden agregar nuevos proyectos para pruebas unitarias o integraciones externas.
- *Pruebas Unitarias y Mocking*: Esta división permite realizar pruebas unitarias independientes en cada capa. Los repositorios pueden ser simulados utilizando mocks, facilitando la validación de la lógica de negocio sin necesidad de conectarse a una base de datos real.
- *Modularidad y Mantenibilidad*: La estructura vertical utilizada dentro de LogicaNegocio mejora significativamente la modularidad del proyecto al mantener cada funcionalidad aislada en su propio espacio. Esto no solo facilita la navegación del código, sino que también promueve un desarrollo incremental. De este modo, es posible agregar o modificar funcionalidades sin afectar otras partes del sistema.

Otra decisión clave en nuestro diseño es la utilización del ciclo de vida Scoped para las instancias de las lógicas y repositorios en la aplicación. Al utilizar Scoped, aseguramos que las mismas instancias sean reutilizadas dentro de un mismo contexto, lo que mejora el rendimiento y evita la recreación innecesaria de objetos, como ocurriría si usáramos Transient, que genera una nueva instancia en cada uso. Esto es especialmente útil cuando múltiples componentes necesitan acceder a las mismas instancias de lógica dentro de una operación, asegurando consistencia en los datos y evitando sobrecargas de memoria. Para el acceso a datos, utilizamos Entity Framework con SQL Server. Tenemos un Contexto SQL que actúa como intermediario entre la aplicación y la base de datos. La arquitectura de acceso a datos sigue una separación de responsabilidades entre la lógica de negocio y la capa de persistencia, lo que mejora la mantenibilidad del código. Cada entidad tiene su correspondiente Repositorio, el cual es responsable de las interacciones directas con la base de datos.

En cuanto al manejo de excepciones, hemos implementado un filtro de excepciones personalizado (ExcepcionFiltro), que se añade a nivel global en la configuración del controlador. Este filtro captura y gestiona las excepciones no controladas, permitiendo una respuesta uniforme y clara ante errores inesperados. Al configurar este filtro en un solo lugar, mejoramos la mantenibilidad y garantizamos que los errores se manejen de manera consistente en toda la aplicación.

Principios de Diseño

Con respecto a la decisión estructural de nuestro programa podemos destacar ciertos aspectos teniendo en cuenta los principios SOLID y los patrones GRASP:

Beneficios en términos de principios SOLID:

- Responsabilidad Única (SRP): Cada proyecto tiene un propósito específico, esto mejora la claridad y reduce el acoplamiento.
 - LogicaNegocio: Gestiona las reglas de negocio y validaciones.
 - Persistencia: Maneja el acceso a datos mediante repositorios.
 - WebApi: Se centra en la comunicación con el cliente, actuando como intermediario. Esto asegura que los cambios en una capa no impacten negativamente en las demás.
 - CargarImplementacion: Carga e instanciación de implementaciones concretas de interfaces a partir de ensamblados externos.
- Segregación de Interfaces (ISP): Las interfaces definidas en LogicaNegocio se diseñan específicamente para cumplir las necesidades de cada funcionalidad. Esto evita incluir métodos innecesarios en las interfaces, manteniéndose pequeñas, cohesivas y fáciles de usar.
- Principio de Inversión de Dependencias (DIP): La lógica de negocio depende de interfaces (abstracciones) en lugar de implementaciones concretas. Estas interfaces se implementan en Persistencia, lo que desacopla completamente las reglas de negocio de los detalles de la base de datos, permitiendo que los cambios en la persistencia no afecten al negocio.

- Abierto/Cerrado (OCP): El diseño permite la extensión del sistema sin modificar el código existente. Por ejemplo, es posible añadir nuevos repositorios o funcionalidades sin alterar la lógica de negocio ya implementada, facilitando la evolución del sistema.

Beneficios en términos de Patrones GRASP:

- Alta Cohesión: Cada componente tiene una responsabilidad única y está diseñado para realizar exclusivamente esa tarea. Por ejemplo, las clases de lógica en LogicaNegocio se enfocan en implementar las reglas de negocio, mientras que las clases de persistencia en Persistencia manejan la interacción con la base de datos. Esta alta cohesión mejora la legibilidad y facilita el mantenimiento.
- Controlador: Los controladores en WebApi delegan las tareas complejas a la lógica de negocio en LogicaNegocio. Esto asegura que la API se mantenga ligera, manejando únicamente la interacción con los clientes y dejando las validaciones y reglas al módulo correspondiente.
- Experto: Las clases están diseñadas para ejecutar las tareas relacionadas con la información que conocen mejor. Por ejemplo, los repositorios en Persistencia gestionan el acceso a los datos porque tienen conocimiento sobre cómo interactuar con la base de datos, mientras que las clases de lógica en LogicaNegocio se centran en las reglas y validaciones.

Ahora, hacemos un análisis sobre los principios SOLID y GRASP para el uso de Entity Framework con separación de responsabilidades.

Beneficios en términos de principios SOLID:

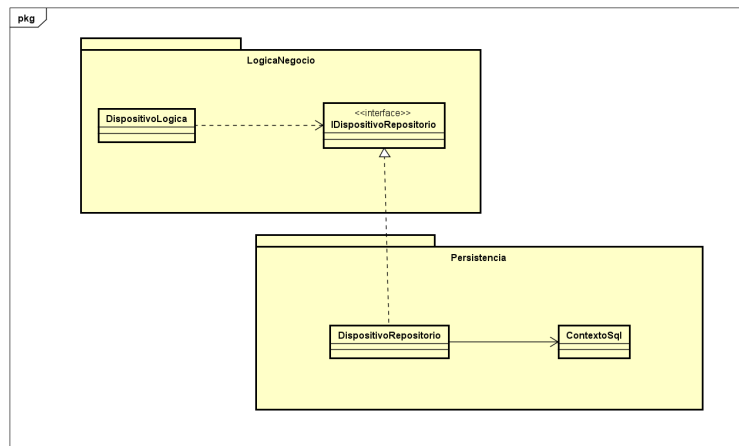
- Principio de Responsabilidad Única (SRP): Cada capa tiene una responsabilidad claramente definida:
 - La lógica de negocio implementa reglas y validaciones.
 - La persistencia maneja la interacción directa con la base de datos a través de Entity Framework.
- Principio de Inversión de Dependencias (DIP): La lógica de negocio depende de interfaces definidas en lugar de implementaciones concretas de Entity Framework, lo que permite reemplazar esta tecnología sin alterar las reglas de negocio.

Beneficios en términos de patrones GRASP:

- Experto: El Contexto SQL encapsula el conocimiento sobre cómo interactuar con la base de datos, mientras que los repositorios manejan las operaciones específicas de cada entidad. Esto respeta el principio de asignar responsabilidades al experto en información.
- Alta Cohesión: Los repositorios son responsables exclusivamente de interactuar con la base de datos, mientras que las clases de lógica se concentran en las reglas de negocio, manteniendo cada componente cohesivo.

Patrones de Diseño

→ Adapter:



El patrón Adapter permite que dos interfaces incompatibles trabajen juntas al traducir las operaciones de una clase en términos que otra pueda entender. En nuestro sistema, utilizamos este patrón para resolver el problema de desacoplar la lógica de negocio de los detalles específicos de acceso a la base de datos. Este desacoplamiento se logra mediante el uso de un repositorio, en este caso `DispositivoRepositorio`, que actúa como intermediario entre la lógica de negocio (`DispositivoLogica`) y la capa de persistencia (representada por `ContextoSql`).

En este caso, utilizamos el patrón Adapter para traducir las operaciones que espera la lógica de negocio (`DispositivoLogica`) en términos que el sistema de persistencia (`ContextoSql`) pueda entender.

La lógica de negocio trabaja con la interfaz `IDispositivoRepositorio`. Dicha interfaz define las operaciones que la lógica espera, y abstrae los detalles de cómo se accede a la base de datos.

La clase `DispositivoRepositorio` actúa como el Adapter. Implementa la interfaz `IDispositivoRepositorio`, lo que le permite a la lógica de negocio interactuar con el repositorio sin conocer los detalles internos de cómo se realizan las operaciones de persistencia. Esta clase traduce las llamadas de `IDispositivoRepositorio` a operaciones compatibles con el contexto `ContextoSql`.

El Adapter (en este caso `DispositivoRepositorio`) traduce las solicitudes de la lógica de negocio en acciones que el contexto de persistencia (`ContextoSql`) puede ejecutar, como buscar un dispositivo en el `DbSet<Dispositivo>`.

`ContextoSql` actúa como el adaptado. Este contexto no implementa directamente la interfaz `IDispositivoRepositorio`, ya que su objetivo es interactuar con la base de datos y no exponer una interfaz específica para la lógica de negocio. Por lo tanto, requiere de una clase adaptadora para que la lógica de negocio no dependa directamente de los detalles de la persistencia.

De esta manera, también utilizamos este patrón para el resto de nuestras lógicas y sus respectivos repositorios.

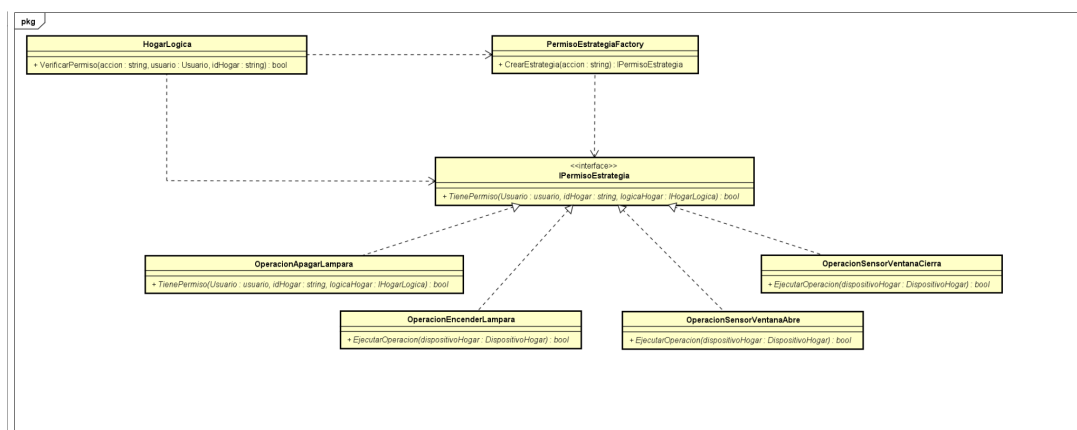
Beneficios en términos de SOLID:

- Principio de Responsabilidad Única (SRP): La lógica de negocio, el repositorio y el contexto de base de datos tienen responsabilidades claramente definidas y separadas.
- Principio Abierto/Cerrado (OCP): Podemos añadir nuevos repositorios sin modificar la lógica de negocio existente, lo que hace que el sistema sea fácilmente extensible.
- Principio de Inversión de Dependencias (DIP): La lógica de negocio no depende directamente de los detalles de la persistencia, sino de una abstracción, lo que garantiza un bajo acoplamiento y facilita los cambios en la persistencia sin afectar al negocio.

También se siguen ciertos patrones GRASP:

- Experto: A través del uso de los repositorios, nos aseguramos que las clases que tienen la información suficiente para realizar una tarea son las que la ejecutan. El DispositivoRepositorio tiene el conocimiento sobre cómo interactuar con la base de datos, mientras que la lógica de negocio (DispositivoLogica) se mantiene enfocada en las reglas de negocio sin preocuparse por los detalles de persistencia.
- Bajo Acoplamiento: Utilizando el patrón Adapter, la lógica de negocio queda desacoplada de los detalles de persistencia. Este bajo acoplamiento facilita el mantenimiento y la evolución del sistema.
- Alta Cohesión: Cada clase tiene una responsabilidad claramente definida. DispositivoLogica se encarga de la lógica de negocio, mientras que DispositivoRepositorio se encarga de la interacción con la base de datos. Esto refuerza la cohesión en cada componente y facilita la mantenibilidad.

→ Strategy con Fábrica:



Identificamos que algunas de nuestras operaciones podrían ser manejadas de manera más eficiente. Con el objetivo de mejorar la estructura y facilitar la evolución del sistema, se

decidió aplicar patrones de diseño que permitieran manejar estas operaciones de manera eficiente y flexible.

Para resolver esta necesidad, se optó por utilizar dos patrones de diseño fundamentales: Strategy y Factory. Estos patrones fueron elegidos específicamente para gestionar las operaciones que deben ser persistidas en la base de datos sobre los dispositivos y gestionar los permisos de los usuarios sobre sus hogares.

A través de estos patrones, el sistema puede manejar dichas operaciones de forma desacoplada, extensible y fácil de mantener, lo que favorece el cumplimiento de los principios SOLID y asegura que el código sea flexible a futuras modificaciones o adiciones.

A continuación, se detalla cómo los patrones utilizados contribuyen a obtener un diseño más mantenible y extensible. Los ejemplos brindados serán orientados hacia la gestión de operaciones de los dispositivos, ya que para la gestión de los permisos de los usuarios se realiza de forma muy similar.

1. Patrón Strategy

La interfaz `IDispositivoOperacion` actúa como la clase Strategy, y las implementaciones concretas como `OperacionApagarLampara`, `OperacionEncenderLampara`, `OperacionSensorVentanaAbre` y `OperacionSensorVentanaCierra` son las ConcreteStrategies.

Extensibilidad: Agregar nuevas operaciones de dispositivo (como encender o apagar otros tipos de dispositivos) se puede hacer implementando nuevas clases que extienden la interfaz `IDispositivoOperacion`. Esto se realiza sin modificar el código existente, cumpliendo con el principio Open/Closed.

Mantenimiento: La lógica específica de cada operación está encapsulada en su propia clase, lo que reduce la complejidad de la clase `DispositivoHogarLogica` y mejora la legibilidad.

2. Fábrica para creación de estrategias

`DispositivoOperacionFactory` se encarga de crear instancias de las estrategias de operación según el tipo de dispositivo y la acción solicitada.

Centralización de la creación de instancias: `DispositivoOperacionFactory` facilita la creación de objetos y delega la lógica de selección de estrategia, haciendo que la clase `DispositivoHogarLogica` sea más limpia y fácil de mantener.

Reducción de acoplamiento: La lógica de negocio no necesita conocer los detalles de las implementaciones específicas, cumpliendo con el principio de Dependency Inversion al depender de abstracciones en lugar de implementaciones.

Beneficios en términos de SOLID:

- (SRP): Cada clase tiene una única razón para cambiar. Las clases como OperacionEncenderLampara y OperacionApagarLampara tienen la responsabilidad de definir la lógica específica de una operación de dispositivo.
- (OCP): El sistema está abierto a la extensión, pero cerrado a la modificación. Se pueden agregar nuevas operaciones de dispositivos sin modificar las clases existentes, solo creando nuevas implementaciones de IDispositivoOperacion y agregándolas a la lógica de la fábrica.
- (LSP): Las instancias de IDispositivoOperacion pueden ser intercambiadas sin afectar el comportamiento de DispositivoHogarLogica, garantizando que todas las estrategias implementan el mismo contrato. Esto garantiza que, en cualquier momento, el sistema puede intercambiar una operación por otra sin romper la lógica.
- (ISP): La interfaz IDispositivoOperacion es pequeña y específica, cumpliendo con este principio al definir solo el método EjecutarOperacion, lo que permite a cada clase implementar solo lo que necesita.
- (DIP): DispositivoHogarLogica depende de la abstracción IDispositivoOperacion y no de implementaciones concretas, permitiendo una mayor flexibilidad y desacoplamiento.

Impacto en la Mantenibilidad del Sistema

El uso de estos patrones y la aplicación de los principios SOLID han mejorado la estructura y mantenibilidad del sistema de las siguientes formas:

Flexibilidad y extensión: Si más tarde se desea agregar nuevos tipos de dispositivos o nuevas operaciones, simplemente se agregarían nuevas clases de operaciones sin modificar el código existente.

Desacoplamiento: El controlador que maneja las operaciones de los dispositivos no necesita conocer los detalles específicos de cada operación. Solo necesita saber cómo interactuar con la interfaz IDispositivoOperacion.

En conclusión, la aplicación de los patrones Strategy y Factory, junto con la adherencia a los principios SOLID, ha resultado en un diseño más robusto, mantenible y extensible para las operaciones de dispositivos en mi proyecto.

Mecanismos utilizados para la extensibilidad solicitada

Asignar nombre al hogar: Nuevo atributo Alias (nulleable) en Hogar. Al momento de crear un nuevo hogar, se puede especificar cuál será dicho atributo, o en caso de no querer hacerlo, el dueño del hogar podrá asignarle un nombre a su hogar más adelante. Se declaró nulleable para que al momento de crear un hogar no sea necesario especificar el alias.

Nombre custom a los dispositivos: Nuevo atributo Nombre en DispositivoHogar. Al momento de asignar un dispositivo a un hogar, dicho atributo se carga con el nombre del dispositivo. Luego, un miembro del hogar con el permiso PermisoModificarNombreDispositivo podrá personalizar el atributo Nombre mencionado anteriormente.

Administradores y dueños de empresa como dueño de hogar: A los roles Admin y DueñoEmpresa, se le agregó un nuevo permiso para poder funcionar como dueños de hogar. Creamos dos nuevos roles predefinidos, uno que agrupa los permisos de administrador con dueño de hogar, y otro que agrupa los permisos de dueño de empresa con dueño de hogar. En la lógica de usuario, solamente tuvimos que agregar un método para realizar dicho cambio de rol (en caso de tener el permiso anteriormente mencionado), y en el repositorio hicimos un método para que dicha actualización de rol se refleje en la base de datos.

Selección de lógica de validación de modelo: Utilizando la interfaz proporcionada en Aulas, creamos dos nuevos proyectos que la implementan, por lo que tienen el método EsValido. Dicho método fue utilizado en los proyectos para validar que un modelo de dispositivo tenga 6 letras o 3 letras seguidas de 3 números. Luego generamos ensamblados a partir de estos proyectos, colocando las dll obtenidas dentro de la carpeta “Validadores en nuestro repositorio”.

Además, creamos un nuevo paquete SmartHome.CargarImplementacion en nuestra solución para poder manejar dinámicamente las dll que se encuentren dentro de una carpeta específica.

Soporte de dos nuevos tipos de dispositivos: Para este requerimiento, únicamente tuvimos que agregar los enums correspondiente a los nuevos tipos en nuestro archivo TipoDispositivo.cs. Luego, agregamos 2 nuevos métodos dentro de la lógica para crear nuevos dispositivos con los nuevos tipos. Esto podría haber sido más fácil de implementar y más extensible a futuro si en la lógica tuviéramos un único método que reciba el tipo de dispositivo a crear por parámetro.

Listado de dispositivos de un hogar: Como dentro de Hogar ya teníamos una lista con sus dispositivos, realizar este requerimiento no supuso ninguna dificultad adicional. Únicamente tuvimos que agregar un método en el controlador de hogares para listar dichos dispositivos.

Agrupar dispositivos por cuarto: Este requerimiento fue más costoso que los anteriores ya que se introduce el concepto de cuarto que antes no era requerido, aunque no supuso grandes dificultades. Creamos una nueva entidad Cuarto con su respectiva lógica y repositorio. Dentro de Cuarto, agregamos vínculos hacia Hogar y DispositivoHogar con los que Entity Framework se encarga de realizar las referencias.

Además, creamos un nuevo permiso para los miembros de los hogares, indicando si pueden administrar o no los cuartos de un hogar.

Importar dispositivos inteligentes: Este requerimiento fue realizado de forma similar al de validación de modelos, con la dificultad agregada de que la interfaz no fue proporcionada, sino que fue desarrollada por nuestra cuenta.

Dicha Interfaz quedó de la siguiente manera:

```
public interface IImportador
{
    0 references
    List<DispositivoDto> Importar(string rutaArchivo);

    0 references
    bool EsArchivoValido(string rutaArchivo);
}
```

```
2 references
public record class FotoDto
{
    0 references
    public string Url { get; init; } = null!;
    0 references
    public bool EsPrincipal { get; init; }
}
```

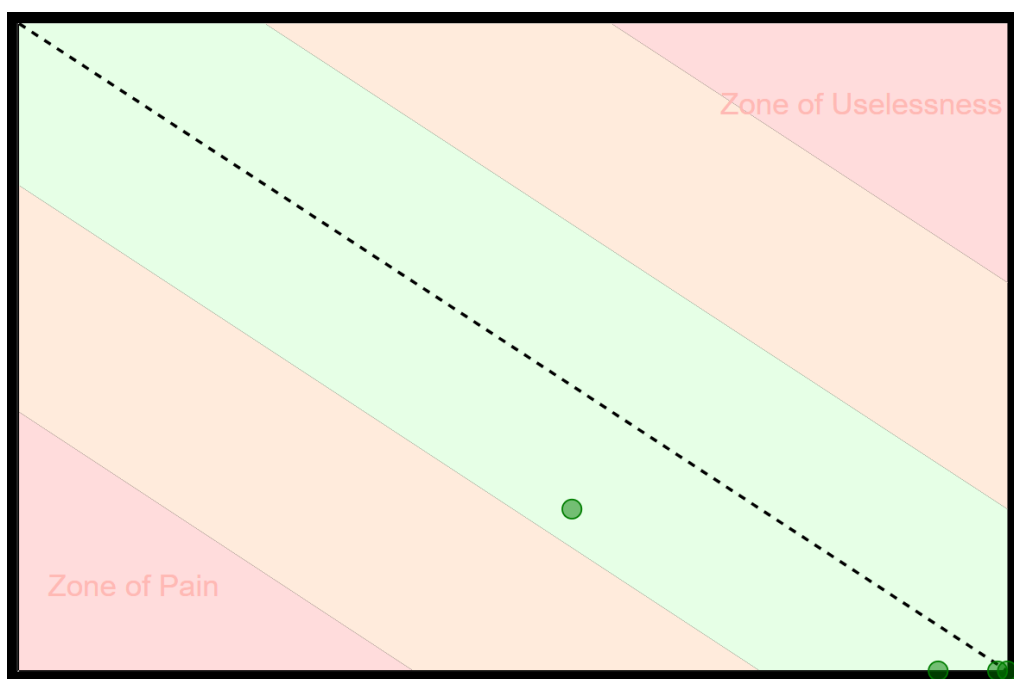
```
public record class DispositivoDto
{
    public string Id { get; set; } = null!;
    public string Tipo { get; set; } = null!;
    public string Nombre { get; set; } = null!;
    public string Modelo { get; set; } = null!;
    public string Descripcion { get; init; } = null!;
    public List<FotoDto> Fotografias { get; set; } = new List<FotoDto>();
    public bool? DetectaPersona { get; set; }
    public bool? DetectaMovimiento { get; set; }
    public bool? UsoExterior { get; set; }
    public bool? UsoInterior { get; set; }
}
```

Luego, creamos un proyecto que implementa dicha interfaz, donde realizamos la lógica para la importación del archivo .json de Aulas. Para esto, tuvimos que deserializar los datos recibidos y convertirlos a una lista de DispositivoDto, tal como lo declara la interfaz.

En la lógica de dispositivos, agregamos un nuevo método para realizar la importación, que a través del paquete SmartHome.CargarImplementacion, realiza importaciones a partir de las dll ubicadas en la carpeta “Importadores”. En este método, recibimos una lista de DispositivoDto (la que retorna la interfaz) y convertimos cada elemento en un Dispositivo, para que pueda ser manejado dentro de nuestra aplicación.

Métricas

Utilizamos la herramienta NDepend para obtener las métricas de diseño asociadas a nuestro trabajo. La gráfica de abstracción vs inestabilidad obtenida es la siguiente:



Luego, ejecutando una consulta SQL pudimos obtener una tabla con todas las métricas necesarias para realizar nuestro análisis.

```
from t in Assemblies
where t.Abstractness >= 0 && t.Instability >=0
select new { t, t.Abstractness, t.Instability, t.RelationalCohesion, t.DistanceFromMainSeq }
```

4 assemblies	Abstractness	Instability	Relational cohesion	Dist from main seq
4 assemblies matched				
SmartHome.CargarImplementacion	0	0.93	1	0.047
SmartHome.LogicaNegocio	0.25	0.56	3.16	0.14
SmartHome.Persistencia	0	0.99	2.01	0.0057
SmartHome.WebApi	0	1	1.17	0

Evaluación de Principios:

Abstracciones Estables: Los módulos más abstractos (alta abstracción) también son más estables (baja inestabilidad). Esto significa que los módulos centrales del sistema son resilientes y no dependen de cambios frecuentes.

En nuestro caso, se ve como nuestro módulo con mayor abstracción (LogicaNegocio), también es el que tiene mayor estabilidad. Además, el resto de nuestros paquetes tienen abstracción 0, y son altamente inestables, por lo que se cumple el principio.

Dependencias Estables: Este principio establece que los módulos menos estables deben depender de módulos más estables. Sin embargo, en este caso, LogicaNegocio ($I = 0.56$) depende de CargarImplementacion ($I = 0.93$), lo que rompe la regla. Esta excepción se debe a que LogicaNegocio depende de CargarImplementacion para resolver dinámicamente ciertas dependencias en tiempo de ejecución. Esta dependencia es consciente y necesaria para mantener la extensibilidad y flexibilidad del sistema, lo que compensa el desajuste con el principio.

En cuanto al resto de dependencias, se alinean correctamente en base al principio.

Clausura Común: Los paquetes que agrupan clases con cambios relacionados parecen haber sido diseñados correctamente, dado que no hay alta inestabilidad en módulos que deberían permanecer cerrados ante cambios. Esto minimiza el riesgo de propagación de errores.

Reuso Común: Los paquetes analizados muestran un diseño modular que respalda este principio. Las clases agrupadas en un mismo paquete tienen responsabilidades coherentes y específicas, lo que permite su reuso conjunto sin arrastrar dependencias innecesarias.

Análisis de métricas:

Cohesión Relacional: El análisis de cohesión relacional destaca dos paquetes con valores inferiores al umbral de 1.5, lo que no es óptimo.

CargarImplementacion (Cohesión: 1.0):

Contiene una única clase, justificada por su responsabilidad aislada y poco cohesiva con otros paquetes.

WebApi (Cohesión: 1.17):

Formado principalmente por controladores que operan de manera independiente, lo que explica el bajo valor de cohesión.

Ambos paquetes reflejan decisiones arquitectónicas conscientes que no afectan la calidad general del diseño.

Distancia, Inestabilidad y Abstracción: La gráfica de abstracción vs. inestabilidad muestra que los paquetes analizados están ubicados en la zona verde, cerca de la secuencia principal (baja distancia a la línea diagonal). Esto indica que el diseño cumple con un buen balance entre abstracción (qué tan genérico o extendible es un módulo) e inestabilidad (qué tan dependiente es de otros).

Los paquetes más abstractos están diseñados para ser extensibles y reutilizables, mientras que los paquetes concretos, con baja abstracción y alta inestabilidad, funcionan como componentes funcionales específicos. Esto demuestra un diseño bien alineado con los principios de arquitectura.

Conclusiones

El sistema mantiene un balance adecuado entre abstracción y estabilidad. Los módulos abstractos son estables, mientras que los concretos son inestables, ubicándose correctamente en la secuencia principal y evitando las "Zonas de Dolor" o "Inutilidad". Esto valida un diseño modular, extensible y robusto.

Aunque algunos paquetes presentan valores bajos de cohesión, estas decisiones están justificadas por sus responsabilidades específicas y no comprometen la calidad general del diseño.

La arquitectura logra cumplir mayoritariamente con los principios de Abstracciones Estables, Clausura Común, y Reuso Común, permitiendo un sistema modular y mantenible. La única excepción detectada está bien justificada y no afecta negativamente la calidad del diseño.

Esto asegura una arquitectura que facilita el mantenimiento y evolución del sistema.

Mejoras a destacar con respecto a la primer entrega

Para esta segunda entrega, implementamos una serie de mejoras significativas en comparación con lo entregado anteriormente. Estas mejoras no solo refuerzan la funcionalidad y la estabilidad del sistema, sino que también introducen elementos clave que optimizan su arquitectura y amplían sus capacidades. A continuación destacamos las 4 principales mejoras de nuestro proyecto:

- 1) **Reutilización de código:** Identificamos que varias funciones en el sistema realizaban tareas similares o repetitivas. Para optimizar esto, implementamos funciones auxiliares que encapsulan la lógica compartida, permitiendo que sean reutilizadas por otras partes del código. Este enfoque no solo reduce la duplicación de código, sino que también mejora la mantenibilidad al centralizar la lógica común en un solo lugar y hace que cada método tenga una responsabilidad más clara y limitada, mejorando en términos de SRP. Además, facilita el seguimiento de errores, aumentando la consistencia y reduciendo el riesgo de inconsistencias.
- 2) **Limpieza de lógica y validaciones en los controladores:** En esta entrega, trasladamos la lógica y las validaciones que inicialmente se encontraban en los controladores hacia las clases de lógica de negocio. Esto permitió simplificar los controladores, limitando su responsabilidad a gestionar las solicitudes y delegar las operaciones al nivel adecuado. Esta separación mejora la claridad del código, facilita la identificación y corrección de errores, y promueve una arquitectura más modular y alineada con el principio de responsabilidad única (SRP). Además, al centralizar las validaciones y la lógica en un único lugar, generamos un sistema más coherente y fácil de mantener a medida que crece en funcionalidad.
- 3) **Utilización del patrón Strategy:** Como mencionamos anteriormente, implementamos el patrón Strategy para gestionar operaciones relacionadas con los dispositivos, definiendo una interfaz genérica (`IDispositivoOperacion`) y estrategias concretas como `OperacionApagarLampara` y `OperacionSensorVentanaAbre`. Este diseño mejora la modularidad y facilita la extensibilidad, permitiendo añadir nuevas operaciones sin modificar el código existente. Además, asegura un sistema más mantenible, flexible y alineado con los principios SOLID, como argumentamos en la sección de Patrones de Diseño.
- 4) **Utilización del patrón Strategy:** Como también fue mencionado en partes anteriores, implementamos un patrón Strategy similar al de las operaciones de los dispositivos, pero para gestionar las consultas sobre permisos de usuarios en hogares, ofreciendo una interfaz genérica (`IPermisoEstrategia`) y estrategias concretas como `PermisoAdministrarCuartoEstrategia` o `PermisoListarMiembroEstrategia`. De igual manera, esto significa una mejora en todos los principios SOLID.

Anexo

Descripción de los resources de la API

Desarrollamos nuestra Api con diferentes resources.

Administradores

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un administrador.	POST	/administradores	Token de administrador.	Nombre: <i>string</i> Apellido: <i>string</i> Email: <i>string</i> Contraseña: <i>string</i>	200 400 401 403 500
Eliminar un administrador.	DELETE	/administradores /{email}	Token de administrador.		200 400 401 403 500

Cámaras

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear una cámara.	POST	/camaras	Token de dueño de empresa.	Nombre: <i>string</i> Modelo: <i>string</i> Descripcion: <i>string</i> fotografias: <i>List<Fotografias></i> [{ Url : <i>string</i> EsPrincipal: <i>bool</i> }] DetectaMovimiento: <i>bool</i> DetectaPersona: <i>bool</i> UsoInterior:	200 400 401 403 500

				<i>bool</i> UsoExterior: <i>bool</i>	
--	--	--	--	--	--

Cuartos

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Agrega un dispositivo de un hogar a un cuarto de dicho hogar.	POST	/cuartos/{id}/dispositivos-hogar	Token	DispositivoHogarId: <i>string</i>	200 400 401 500

Dispositivos

DESCRIPCIÓN	MÉTODO	URI	QUERY	HEADER	BODY	RESPUESTA
Obtener un listado de todos los dispositivos registrados.	GET	/dispositivos	NumeroDePagina: int (si <= 0 se le asigna 1) TamañoDePagina: int (si <= 0 se le asigna 10) NombreDispositivo: string Modelo: string NombreEmpresa: string TipoDispositivo: string	Token		200: [{ Nombre: <i>string</i> NumeroModelo: <i>int</i> FotoPrincipal: <i>string</i> NombreEmpresa: <i>string</i> }] 401 500
Obtener un	GET	/dispositivos/		Token		200:

listado de todos los tipos de dispositivos inteligentes que soportan el sistema.		tipos				<pre>[{ Tipo: string }]</pre> 401 500
--	--	-------	--	--	--	---

Dispositivos Hogar

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Un sensor de un hogar se abre y genera notificación.	POST	/dispositivos-hogar/{id}/abrir			200 400 500
Un sensor de un hogar se cierra y genera notificación.	POST	/dispositivos-hogar/{id}/cerrar			200 400 500
Un sensor de un hogar detecta movimiento y genera notificación.	POST	/dispositivos-hogar/{id}/movimiento-sensor			200 400 500
Un dispositivo de un hogar se pone en línea.	POST	/dispositivos-hogar/{id}/conectar	token		200 400 401 500
Un dispositivo de un hogar deja de estar en línea.	POST	/dispositivos-hogar/{id}/desconectar	token		200 400 401 500
Una cámara de un hogar detecta movimiento y genera notificación.	POST	/dispositivos-hogar/{id}/movimiento-camera			200 400 500
Una cámara de un hogar detecta a una persona y	POST	/dispositivos-hogar/{id}/persona			200 400 500

genera notificación.					
Una lámpara de un hogar se enciende y genera notificación.	POST	/dispositivos-hogar/{id}/encender			200 400 500
Una lámpara de un hogar se apaga y genera notificación.	POST	/dispositivos-hogar/{id}/apagar			200 400 500
Se modifica el nombre de un dispositivo de un hogar.	PATCH	/dispositivos-hogar/{id}/nombre	token		200 400 401 500

Dueños Empresa

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un dueño de empresa.	POST	/dueños-em presa	Token de administrador	Nombre: <i>string</i> Apellido: <i>string</i> Email: <i>string</i> Contraseña: <i>string</i>	200 400 401 403 500

Dueños Hogar

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un dueño de hogar.	POST	/dueños-hogar		Nombre: <i>string</i> Apellido: <i>string</i> Email: <i>string</i>	200 400 500

				Contraseña: <i>string</i> FotoPerfil: <i>string</i>	
Obtiene los permisos de un dueño hogar sobre determinado hogar.	GET	/dueños-hogar /{id_hogar}/permisos	token		200 [PermisoAsociarDispositivos: <i>bool</i> PermisoListarDispositivos: <i>bool</i> PermisoAdministrarCuartos: <i>bool</i> PermisoModificarNombreDispositivos: <i>bool</i> PermisoAgregarMiembros: <i>bool</i> PermisoListarMiembros: <i>bool</i> PermisoModificarAlias: <i>bool</i>] 400 500

Empresas

DESCRIPCIÓN	MÉTODO	URI	QUERY	HEADER	BODY	RESPUESTA
Crear empresa	POST	/empresas		Token de dueño empresa.	Nombre: <i>string</i> Logotipo: <i>string</i> Rut: <i>string</i>	200 400 401 403 500

Listar empresas	GET	/empresas	NumeroDe Pagina: <i>int</i> (si <= 0 se le asigna 1) TamañoDe Pagina: <i>int</i> (si <= 0 se le asigna 10) Nombre Empresa: <i>string</i> Nombre Completo Dueño: <i>string</i>	Token de administrador		200 [{ Nombre: <i>string</i> Logotipo: <i>string</i> Rut: <i>string</i> NombreDueño: <i>string</i> }] 401 403 500
Listar empresas	GET	/empresas/validadores		Token de dueño empresa.		200 [{ Validador: <i>string</i> }] 400 401 403 500

Hogares

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un hogar	POST	/hogares	Token de dueño hogar	Calle: <i>string</i> NumeroPuerta: <i>int</i> Latitud: <i>int</i> Longitud: <i>int</i> CantMiembros Soportados: <i>int</i>	200 400 401 403 500

Agregar miembro a un hogar	POST	/hogares/{id}/miembros	Token	Email: <i>string</i> PermisoAsociar Dispositivos: <i>bool</i> PermisoListar Dispositivos: <i>bool</i> PermisoNotificaciones: <i>bool</i> PermisoAdministrarCuartos: <i>bool</i> PermisoModificarNombreDispositivos: <i>bool</i>	200 400 401 403 500
Asociar un dispositivo a un hogar.	POST	/hogares/{id}/dispositivos	Token	DispositivoId: <i>string</i>	200 400 401 403 500
Listar todos los dispositivos asociados a un hogar.	GET	/hogares/{id}/dispositivos	Token		200: [{ Nombre: <i>string</i> NumeroModelo: <i>int</i> FotoPrincipal: <i>string</i> Nombre Empresa: <i>string</i> EstaConectado: <i>bool</i> EstaAbierto: <i>bool</i> EstaEncendida: <i>bool</i> }] 400 401 403 500

Listar todos los miembros asociados a un hogar.	GET	/hogares/{id}/miembros	Token		200 [NombreCompleto: string Email: string FotoPerfil: string TienePermisoListarDispositivos: bool TienePermisoAsociarDispositivos: bool RecibeNotificaciones: bool TienePermisoAdministrarCuartos: bool TienePermisoModificarNombreDispositivos: bool }] 400 401 403 500
Modificar el alias de un hogar.	PATCH	/hogares/{id}/alias	Token	Alias: string	200 400 401 403 500
Agregar un cuarto a un hogar.	POST	/hogares/{id}/cuartos	Token	Nombre: string	200 400 401 403 500
Listar los hogares de un usuario.	GET	/hogares/usuario	Token		200 [Id: string Email: string Calle: string Alias: string? NumPuerta: int }]

					400 401 403 500
Listar cuartos de un hogar.	GET	/hogares/{id}/cuartos	Token		200 [{ Id: string Nombre: string }] 400 401 403 500

Importadores

DESCRIPCIÓN	MÉTODO	URI	QUERY	HEADER	BODY	RESPUESTA
Importar dispositivos.	POST	/importadores/dispositivos	Tipo Dispositivo: string FechaDe Creacion: DateTime Leida: bool	Token	Ruta: string IdentificadorImportador: string	200 400 401 403 500
Obtener los identificadores de los importadores existentes.	GET	/importadores				200 [{ Importador: string }] 400 500

Lámparas

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear una lámpara.	POST	/lámparas	Token de dueño de empresa.	Nombre: <i>string</i> Modelo: <i>string</i> Descripcion: <i>string</i> fotos: <i>List<Fotografias></i> [[Url : <i>string</i> EsPrincipal: <i>bool</i>]]	200 400 401 403 500

Notificaciones

DESCRIPCIÓN	MÉTODO	URI	QUERY	HEADER	BODY	RESPUESTA
Listar notificaciones	GET	/notificaciones	Tipo Dispositivo: <i>string</i> FechaDe Creacion: <i>DateTime</i> Leida: <i>bool</i>	Token		200 [[Evento: <i>string</i> FueLeida: <i>bool</i> FechaHora: <i>string</i> NombreDispositivoHogar: <i>string</i>]] 401 500
Marcar las notificaciones de un usuario como	PATCH	/notificaciones/leidas	Tipo Dispositivo: <i>string</i>	Token		200 401 500

leídas.			FechaDe Creacion: <i>DateTime</i> Leida: <i>bool</i>			
---------	--	--	---	--	--	--

Sensores

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Crear un sensor de movimiento.	POST	/sensores-movimiento	Token de dueño empresa.	Nombre: <i>string</i> NumeroModelo: <i>int</i> Descripcion: <i>string</i> fotografias: <i>List<Fotografias></i> [[Url: <i>string</i> EsPrincipal: <i>bool</i>]]	200 400 401 403 500
Crear un sensor de ventana.	POST	/sensores-ventana	Token de dueño empresa.	Nombre: <i>string</i> NumeroModelo: <i>int</i> Descripcion: <i>string</i> fotografias: <i>List<Fotografias></i> [[Url: <i>string</i> EsPrincipal: <i>bool</i>]]	200 400 401 403 500

Sesiones

DESCRIPCIÓN	MÉTODO	URI	HEADER	BODY	RESPUESTA
Utilizado para obtener un token de la sesión de un usuario.	POST	/sesiones		Email: <i>string</i> Contraseña: <i>string</i>	200 [Token: <i>string</i> Permisos: List<String>] 400 500
Utilizado para eliminar la sesión de un usuario.	DELETE	/sesiones	Token		200 400 401 500

Usuarios

DESCRIPCIÓN	MÉTODO	URI	QUERY	HEADER	BODY	RESPUESTA
Obtener un listado con todos los usuarios registrados.	GET	/usuarios	NumeroDePagina: <i>int</i> (si <= 0 se le asigna 1) TamañoDePagina: <i>int</i> (si <= 0 se le asigna 10) Rol: <i>string</i> NombreCompleto: <i>string</i>	Token de administrador.		200 [Nombre: <i>string</i> Apellido: <i>string</i> NombreCompleto: <i>string</i> TipoRol: <i>string</i> FechaCreacion: <i>DateTime</i>] 401 403 500

Agregar permisos de dueño hogar a un usuario.	PATCH	/usuarios /permisos		Token		200 [Permisos: List<string>] 401 403 500
Cambiar la foto de perfil de un usuario.	PATCH	/usuarios /foto-perfil		Token	FotoPerfil: string	200 401 403 500

El Token que se incluye en el Header al realizar estos endpoints debe pertenecer a una sesión autenticada, en la cual el usuario logueado tenga una sesión activa en la lista de sesiones. Al desautenticarse, el token deja de ser válido, por lo que el usuario deberá autenticarse nuevamente para ejecutar acciones que requieran header.

Cobertura de Test Unitarios

Para nuestro proyecto utilizamos Test Unitarios con implementación de Mocks para validar verdaderamente una única funcionalidad. Gracias a este objetivo, implementamos una totalidad de 488 pruebas. Para esta entrega se agregaron 153 tests. Estos cubren la gran mayoría de casos posibles a los que puede verse enfrentado el sistema mientras es utilizado.

Paquete Persistencia

Este paquete tiene una baja cobertura debido a todas las migraciones que se hacen dentro del mismo. Estas no son testeadas, pero si se toman en cuenta a la hora de calcular el porcentaje de cobertura. Sin contar esto, logramos obtener el 100% de cobertura para todas las clases implementadas en este paquete.

SmartHome.Persistencia	5%	5815/6114
SmartHome.Persistencia	5%	5815/6114
ContextoSql	100%	0/53
CuartoRepositorio	100%	0/14
DispositivoHogarRepositorio	100%	0/25
DispositivoRepositorio	100%	0/45
EmpresaRepositorio	100%	0/33
HogarRepositorio	100%	0/30
NotificacionRepositorio	100%	0/31
SesionRepositorio	100%	0/18
UsuarioRepositorio	100%	0/50
Migrations	0%	5815/5815

Paquete LogicaNegocio

Dentro del paquete Logica Negocio logramos cubrir prácticamente todas las líneas de código. Esto debido al uso de TDD a lo largo de todo el desarrollo del mismo, además de la constante revisión de la cobertura apenas terminamos de implementar código nuevo.

SmartHome.LogicaNegocio	98%	24/1439
SmartHome.LogicaNegocio	98%	24/1439
Usuarios	100%	0/268
Sesiones	100%	0/41
Cuartos	100%	0/55
Notificaciones	99%	1/95
Dispositivos	99%	3/304
Hogares	98%	6/296
Fabrica	100%	0/11
Estrategia	100%	0/21
Entidades	99%	1/84
HogarLogica	97%	5/180
DispositivosHogar	98%	5/220
Empresas	97%	4/141
ParametroPaginacion	74%	5/19

Paquete WebApi

Al igual que el paquete anterior, en este también logramos cubrir un muy buen porcentaje del código.

SmartHome.WebApi	96%	35/967
SmartHome.WebApi	96%	34/958
Filtros	100%	0/93
Controllers	96%	34/865
Sesiones	100%	0/12
Sensores	100%	0/51
Lamparas	100%	0/44
Importadores	100%	0/23
DueñosHogar	100%	0/77
DueñosEmpresa	100%	0/33
DispositivosHogar	100%	0/59
Cuartos	100%	0/12
Camaras	100%	0/52
Administradores	100%	0/37
Empresas	99%	1/73
Usuarios	98%	1/53
Dispositivos	97%	2/60
Hogares	90%	22/222
Autenticaciones.Modelos	86%	4/28
Notificaciones	86%	4/29

Paquete CargarImplementacion

En este paquete se logró la máxima cobertura posible, evidenciando la efectividad de las pruebas realizadas.

SmartHome.CargarImplementacion	100%	0/32
SmartHome.CargarImplementacion	100%	0/32

Instalación

Se limpian los recursos no utilizados, tanto en network como en system ejecutando los siguientes comandos:

- `docker network prune`
- `docker rm -f `docker ps -q -a``
- `docker system prune`

Luego, para asegurarnos de estar en la misma red de trabajo crearemos una red especialmente para el deploy del obligatorio:

- `docker network create da2-network`

SQL

Para la instalación del SQL lo que vamos a hacer es primero descargar la última imagen sql desde azure:

- `docker pull mcr.microsoft.com/azure-sql-edge`

Y lo siguiente será realizar crear un container llamado: 'sql' utilizando la imagen descargada uniendolo a la red creada anteriormente.

- `docker run --network da2-network -e "ACCEPT_EULA=1" -e "MSSQL_SA_PASSWORD=MyPass@word" -e "MSSQL_PID=Developer" -e "MSSQL_USER=SA" -p 1433:1433 -d --name=sql mcr.microsoft.com/azure-sql-edge`

El contenedor debería levantarse correctamente

☐	Name	Image ↑	Status	CPU (%)	Port(s)	Last started	Actions
☐	sql 716bc0896cd6	mcr.microsoft.com/azure-sql-edge	Running	0.46%	1433:1433	12 seconds ago	■ ⋮ 🗑

Nos aseguramos de que esté funcionando usando Azure Data Studio (Password = MyPass@word):

Connection Details

Connection type: Microsoft SQL Server

Input type: ☒ Parameters ☐ Connection String

Server *: 127.0.0.1

Authentication type: SQL Login

User name *: sa

Password:

☒ Remember password

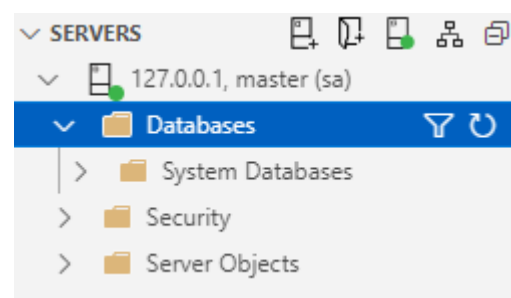
Database: master

Encrypt: ☒ Mandatory

Trust server certificate: ☒ True

Server group: <Default>

Name (optional):



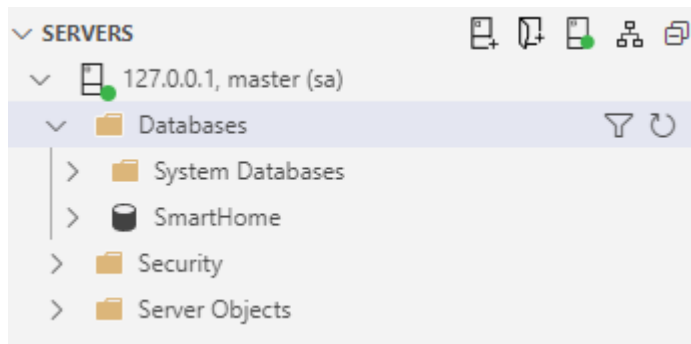
BACKEND

Connection String: "Server = localhost,1433; Database = SmartHome; User Id = sa;
Password = MyPass@word; TrustServerCertificate=True"

Lo siguiente será actualizar mi base datos para replicar todas las migraciones utilizadas, ejecutando el comando sobre 257073_256325_298679:

- `dotnet ef database update --project src/SmartHome.Persistencia --startup-project src/SmartHome.WebApi`

Vemos como se carga nuestra base de datos:



Cambiamos Connection String: "Server = sql,1433; Database = SmartHome; User Id = sa;
Password = MyPass@word; TrustServerCertificate=True"

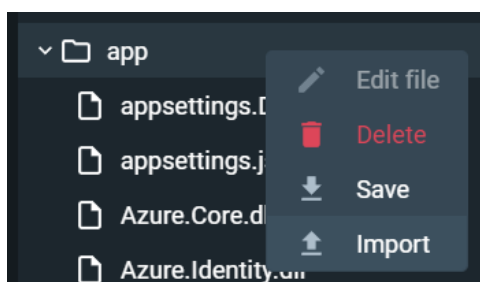
Y procedemos con el deploy ejecutando los siguientes comandos sobre 257073_256325_298679:

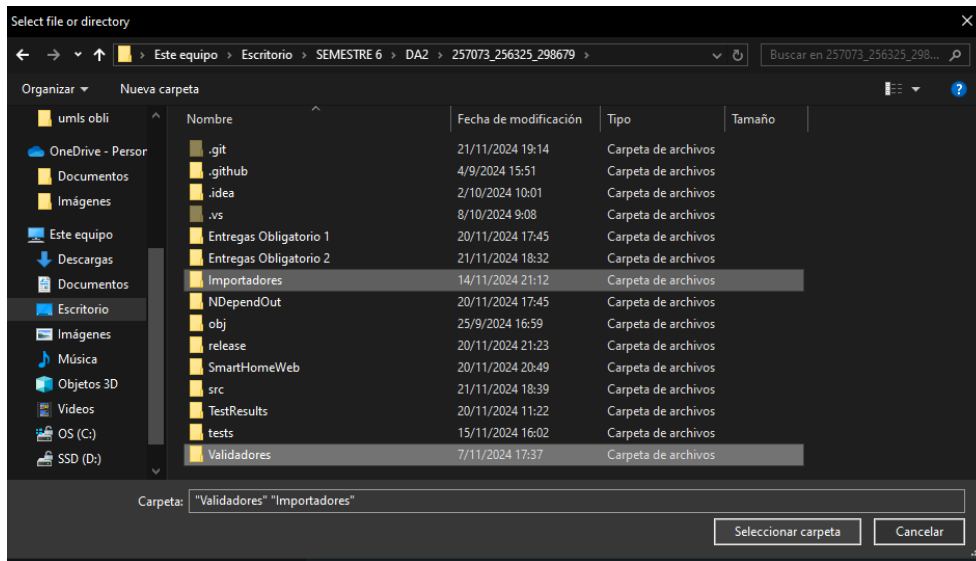
- `docker build --rm -t deploy/api:latest .`
- `docker run --network da2-network --name api --rm -p 5222:5222 -p 5223:5223 -e ASPNETCORE_HTTP_PORT=https://+:5223 -e ASPNETCORE_URLS=http://+:5222 deploy/api:latest`

El contenedor debería levantarse correctamente

☐	Name	Image ↑	Status	CPU (%)	Port(s)	Last started	Actions
☐	api 81123937276b	deploy/api:latest	Running	0%	5222:5222 Show all ports (2)	3 seconds ago	☐ ☐ ☐
☐	sql e08362ff4a91	mcr.microsoft.com/azure-sql-edge	Running	0.45%	1433:1433 Show all ports (2)	24 minutes ago	☐ ☐ ☐

Y dentro de api, en files, nos dirigimos a app y agregamos nuestras carpetas con las dll:





FRONTEND

El puerto declarado en el ambiente debe ser:

```
export default { smartHomeApi: "http://localhost:5222" }
```

Sobre 257073_256325_298679/SmartHomeWeb

- docker build . -t frontend
- docker run --network da2-network --name frontend --rm -p 8080:80 frontend

El contenedor debería levantarse correctamente:

☐	Name	Image	Status	CPU (%)	Port(s)	Last started	Actions
☐	api 81123937276b	deploy/api:latest	Running	0%	5222:5222 Show all ports (2)	3 minutes ago	
☐	frontend c0944d89d1c0	frontend	Running	0%	8080:80	2 seconds ago	
☐	sql e08362ff4a91	mcr.microsoft.com/azure-sql-edge	Running	0.43%	1433:1433	27 minutes ago	

Al entrar al localhost:8080, nuestra aplicación corre correctamente!

